

컴퓨터비전_인공지능(AI)개론

5. 경사하강법(GD) 구현

- 머신러닝 알고리즘의 전체상과 예측 함수
- 신경망의 개념과 파이토치 프로그래밍 모델의 관계
- 파이토치 프로그래밍에 필요한 용어 정의
- 예측 함수의 내부 구조
- 외부에서 본 예측 함수의 거동
- 파이토치 머신러닝 프로그램의 전체상
- 활성화 함수의 목적

멋진 건축물을 지으려면 먼저 설계도를 이해하고, 벽돌을 어떻게 쌓아 올릴지 알아야 합니다.

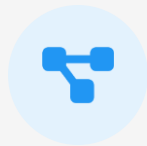
딥러닝 모델을 만드는 것도 이와 같습니다.

우리가 만들고 싶은 인공지능 모델이 '건축물'이라면,

그 핵심 설계도에 해당하는 것이 바로 '예측 함수(Prediction Function)'입니다.

그리고 이 예측 함수를 구성하는 기본 '벽돌'이 바로 파이토치의 '레이어 함수(Layer Function)'입니다.

신경망 개념도



뇌 신경 세포에서 영감을 받은 그림으로, 입력층, 은닉층, 출력층의 동그란 '노드'들이 서로 연결된 형태로 표현됩니다.

손님에게 보여주는 '메뉴판'과 같습니다.

 설계 (Design)

파이토치 프로그램 모델



실제 코딩에 가까운 관점으로, 데이터(텐서)가 어떤 '함수'들을 순서대로 통과하며 변환되는지를 보여줍니다.

주방 내부의 '조리 과정'과 같습니다.

 구현 (Coding)



핵심 포인트: 신경망 그림의 '층(Layer)'은 데이터(텐서)를, 파이토치에서 말하는 '레이어 함수'는 그 데이터를 가공하는 함수를 의미하는 전혀 다른 대상입니다.

CORE CONCEPT

"머신러닝 모델은 레이어 함수(구조)와 파라미터(값)의 유기적인 결합체이다"

레이어 함수

텐서를 입력받아 텐서를 출력하는 함수이자,
모델을 만드는 부품입니다. 예) nn.Linear, nn.ReLU 등

머신러닝 모델

레이어 함수들을 여러 개 조합하여 만든 하나의 거대한 합성 함수
입니다. 입력 텐서를 원하는 출력 텐서로 변환합니다.

파라미터

레이어 함수 내부에 저장되어 학습을 통해 값이 조정되는 데이터
입니다. weight(가중치)와 bias(편향)가 대표적입니다.

학습

모델이 정답에 가까운 출력을 내도록 레이어 함수 내부의 파라미
터 값을 조정하는 행위입니다.

784개의 픽셀 값을 입력받아, 10개(0~9)의 숫자 중 하나로 예측하는 모델을 만들어 보겠습니다.

```
# 첫 번째 선형 함수: 784개 입력을 받아 128개로 출력
```

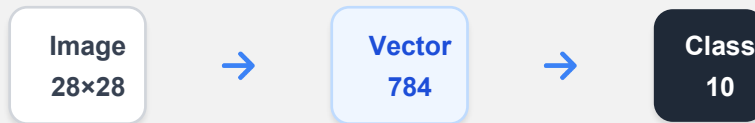
```
l1 = nn.Linear(784, 128)
```

```
# 두 번째 선형 함수: 128개 입력을 받아 10개로 출력
```

```
l2 = nn.Linear(128, 10)
```

```
# 활성화 함수
```

```
relu = nn.ReLU(inplace=True)
```



이미지를 1차원 벡터로 펼쳐서(Flatten) 입력

매우 중요: 첫 번째 레이어 함수의 출력 수(128)와 두 번째 레이어 함수의 입력 수(128)가 정확히 일치해야 합니다.

데이터가 흘러가는 파이프의 크기가 서로 맞아야 하는 것과 같습니다.

1

입력 데이터`torch.Size([100, 784])`

784개의 요소를 가진 데이터가 100개

2

첫 번째 선형 함수`l1(inputs)`

중간 텐서 1 계산

3

활성화 함수`relu(m1)`

중간 텐서 2 계산

4

두 번째 선형 함수`l2(m2)`

출력 텐서 계산

5

최종 출력`torch.Size([100, 10])`

100개 입력 각각에 대해 10개의 예측값

```
# 1. 100개의 데이터, 각 데이터는 784개의 요소를 가짐
inputs = torch.randn(100, 784)
```

```
# 입력 텐서로부터 출력 텐서 계산
# 2. 첫 번째 선형 함수 통과
m1 = l1(inputs) # 중간 텐서 1 계산
```

```
# 3. 활성화 함수 통과
m2 = relu(m1) # 중간 텐서 2 계산
```

```
# 4. 두 번째 선형 함수 통과
outputs = l2(m2) # 출력 텐서 계산
```

```
# nn.Sequential을 사용해 전체를 하나의 합성 함수로 정의
net2 = nn.Sequential(
    l1,
    relu,
    l2
)
# 5. 한 번에 결과 계산
outputs2 = net2(inputs)
```

CORE MESSAGE

"이 세 가지 요소가 없으면 AI 학습은 시작되지 않는다"

01

예측 함수 (net)

학습 데이터를 입력받아
예측값을 출력하는 모델 그 자체입니다.

02

손실 함수 (criterion)

모델의 예측값과 실제 정답을 비교하여 '오차
(손실)'를 계산합니다.

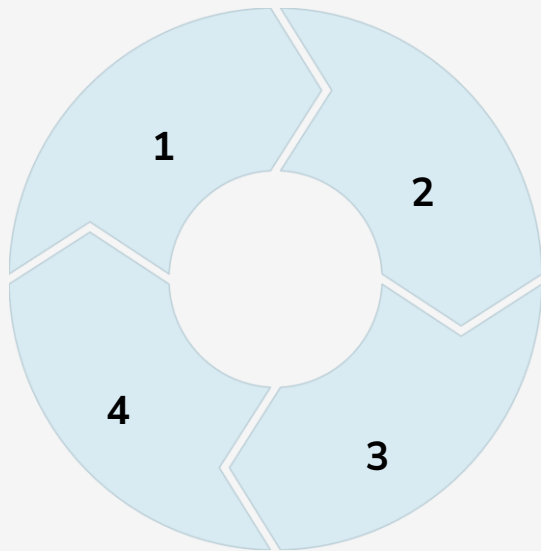
03

최적화 함수 (optimizer)

계산된 손실을 바탕으로 모델의 파라미터를
정답에 더 가까워지는 방향으로 수정합니다.

예측 계산
net(inputs)를 통해 예측값 outputs를
계산합니다.

파라미터 수정
optimizer.step()를 통해 경사값을 바탕
으로 파라미터를 업데이트합니다.



손실 계산
criterion(outputs, labels)를 통해
손실 loss를 계산합니다.

경사 계산
loss.backward()를 통해 각 파라미터가
손실에 얼마나 영향을 미치는지 계산합니
다.

딥러닝 모델을 여러 층으로 쌓아 올린 블록 층이라고 상상해봅시다.

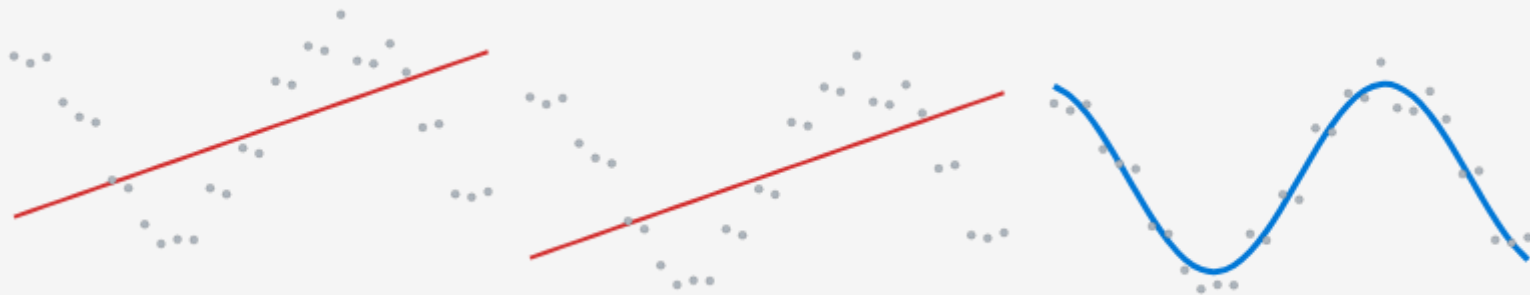
만약 우리가 사용하는 블록이 사각형 모양(선형 함수)뿐이라면,

아무리 높이 쌓아도 결국 전체 모양은 거대한 직육면체를 벗어나지 못할 겁니다.

'활성화 함수'는 바로 이 성에 곡선 모양, 쐐기 모양 등 다양한 형태의 특수 블록(비선형성)을 끼워 넣어 주는 것과 같습니다.

이 특수 블록 덕분에 우리의 모델은 비로소 단순한 직선을 넘어

복잡하고 구불구불한 현실 세계의 데이터를 표현하고 학습할 수 있는 능력을 갖추게 됩니다.



2차 함수 형태의 데이터($y=x^2$)를 학습하는 세 가지 모델을 비교해 보겠습니다:

선형 회귀 모델 (net)

선형 함수 1개로 구성된 가장 단순한 모델

활성화 함수 없는 딥러닝 (net2)

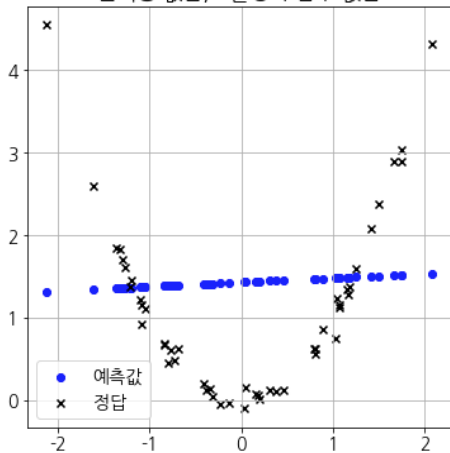
선형 함수 3개를 사용하나
활성화 함수가 없는 모델

올바른 딥러닝 모델 (net3)

선형 함수 사이에
ReLU 활성화 함수를 추가한 모델

선형 회귀 모델 (net)

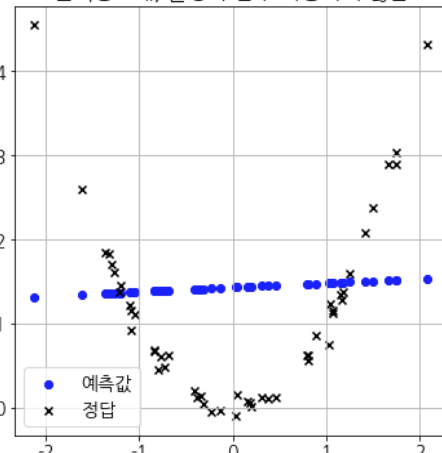
은닉층 없음, 활성화 함수 없음



결과: 직선 형태의 예측만 가능

활성화 함수 없는 딥러닝 (net2)

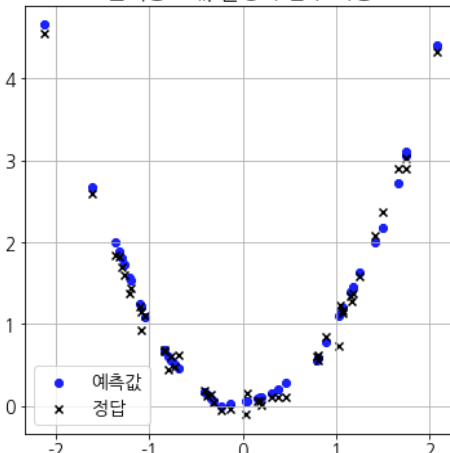
은닉층 2개, 활성화 함수 사용하지 않음



결과: 여전히 직선 형태만 예측

올바른 딥러닝 모델 (net3)

은닉층 2개, 활성화 함수 사용



결과: 2차 함수의 곡선 형태를 성공적으로 학습

실험 결과, 선형 함수를 아무리 여러 번 합성해도 그 결과는 결국 하나의 선형 함수와 같다는 수학적 사실을 확인했습니다.

1

비선형성 부여

활성화 함수는 모델에 '비선형성'을 부여하여 복잡한 패턴을 학습할 수 있게 합니다.

2

층 쌓기의 의미

활성화 함수가 있어야 층을 깊게 쌓는 것이 의미 있어집니다.

3

현실 데이터 학습

구불구불한 현실 세계의 데이터를 표현하고 학습할 수 있는 능력을 제공합니다.

순전파(Forward Pass)

입력 데이터가 신경망의 각 층을 차례로 통과하며 최종 출력을 만들어내는 과정입니다.

마치 공장의 생산 라인처럼,
원자재가 각 공정을 거쳐 최종 제품이 되는 것과 같습니다.

각 층에서는 가중치와 편향을 사용해 입력을 변환하고,
활성화 함수를 거쳐 다음 층으로 전달합니다.

이 과정이 반복되어 최종 예측값이 만들어집니다.

구체적인 계산 흐름

입력 데이터: 집 크기 100m^2

↓

Layer 1: $100 \times 0.5 + 10 = 60$

↓

Layer 2: $60 \times 0.8 + 5 = 53$

↓

출력: 예측 가격 53억원

실제 가격: 50억원

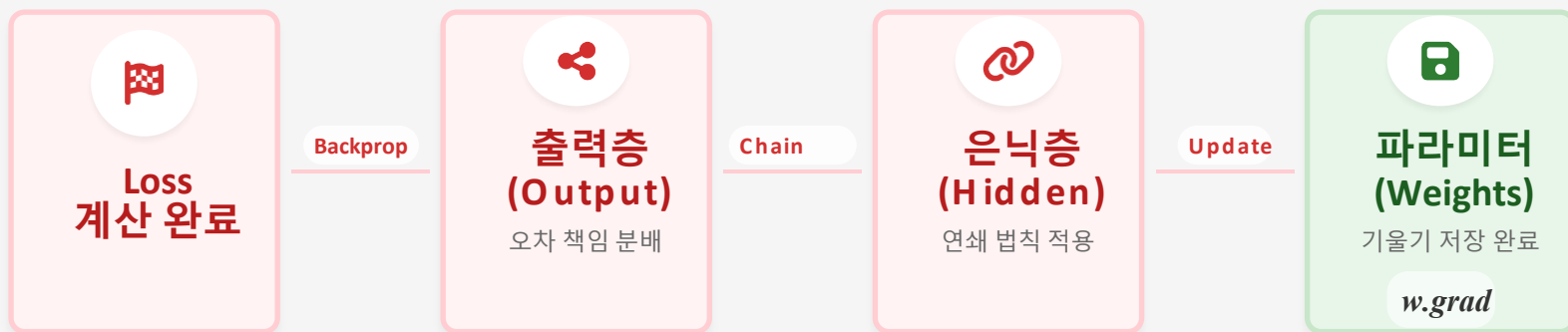
손실: $(53-50)^2 = 9$

각 층마다 중간 결과물이 생성되며, 이는 역전파 단계에서 중요하게 사용됩니다.



예측 오류를 줄이기 위해 각 가중치를 얼마나 수정해야 하는지 계산하는 과정입니다.
최종 손실에서 시작해 거꾸로 올라가며 각 층의 '책임'을 파악합니다





깊은 네트워크에서 역전파 시 그래디언트가 점점 작아지는 현상입니다.
그래디언트(경사)가 0에 가까워짐
소문 전달 게임처럼,
10명을 거치면서 매번 정보의 90%만 전달된다면 결국 35%만 남게 됩니다.

결과: 앞쪽 층들이 거의 학습되지 않아 성능이 크게 저하됩니다.

[비교] 그래디언트 폭발
그래디언트가 매우 큰 값으로 폭발하는 현상입니다.
마치 복리 이자처럼 매일 10% 이자가 붙으면 100일 후 무려 13,780배가 됩니다.

결과: 가중치가 NaN이 되거나 학습이 완전히 발산합니다.

[해결방안] 파이토치 Autograd Hook 으로 실시간 관찰

PyTorch에서 역전파 과정 중 특정 텐서의 그래디언트를 가로채서 확인하거나 수정할 수 있는 기능

마치 CCTV 처럼 각 층마다 CCTV를 설치해서 그래디언트가 어떻게 흘러가는지 실시간 관찰

가준: 평균 -0.01 ~ 0.01 사이 , 표준편차 0.001~0.1 사이 (모든 층에서 비슷한 크기) 좋은 상태로 봄

Hook 함수 정의

```
def gradient_hook(grad):  
    print(f"그래디언트 평균: {grad.mean():.4f}")  
    print(f"그래디언트 표준편차: {grad.std():.4f}")  
    return grad # 수정하지 않고 그대로 반환
```

특정 파라미터에 Hook 등록

```
layer.weight.register_hook(gradient_hook)
```

학습률이란?

가중치를 업데이트할 때 그래디언트를 얼마나 반영할지 결정하는 값

새 가중치 = 기존 가중치 - 학습률 $\times$ 그래디언트

목적지로 가는 보폭에 비유할 수 있습니다.

너무 크면 목적지를 지나치고, 너무 작으면 도착하는 데 오래 걸립니다.

이상적인 전략

- 초반: 큰 학습률로 빠르게 진행
- 후반: 작은 학습률로 정밀하게 조정

1

StepLR - 계단식 감소

일정 에포크마다 학습률을 고정 비율로 감소시킵니다. 예를 들어 10 에포크마다 0.1배로 줄이면 $0.1 \rightarrow 0.01 \rightarrow 0.001$ 로 단계적으로 감소합니다.

장점: 간단하고 직관적

단점: 급격한 변화로 불안정할 수 있음

2

CosineAnnealingLR - 부드러운 곡선

코사인 함수 곡선을 따라 학습률이 자연스럽게 감소합니다. 초반에는 빠르게, 후반에는 매우 천천히 감소하여 정밀한 조정이 가능합니다.

장점: 안정적이고 범용적

적용: 이미지 분류 등에 효과적

3

OneCycleLR - 최신 기법

학습률을 증가시켰다가 다시 감소시키는 전략입니다. Warm-up으로 안정성을 확보하고, 높은 학습률로 빠르게 학습한 뒤, Decay로 정밀하게 마무리합니다.

장점: 가장 효과적인 최신 기법

적용: 대규모 학습에 최적

가중치 초기화의 중요성

- 신경망 학습의 시작점인 가중치 초기화는 학습 성공의 핵심
- 잘못 초기화하면 학습 자체가 불가능해질 수 있음

모든 가중치를 0으로

문제: 모든 뉴런이 똑같이 학습되는 대칭성 문제 발생

3개의 뉴런이 있어도 1개처럼 작동하게 됩니다.

가중치를 너무 크게

문제: 활성화 값이 폭발하고 활성화 함수가 포화 상태에 도달

그래디언트 소실로 학습이 멈춥니다.

가중치를 너무 작게

문제: 10층을 통과하면 $0.001^{10} \approx 0$

신호가 완전히 사라져 정보 전달이 안 됩니다.

Xavier (Glorot)

For Tanh / Sigmoid

Xavier 초기화 (2010)

Tanh와 Sigmoid 같은 대칭 활성화 함수에 최적화된 방법입니다. 입력과 출력 뉴런 개수를 모두 고려하여 가중치의 분산을 조절합니다.

공식: $\sqrt{2/(n_{in} + n_{out})}$

- 각 층의 출력 분산을 입력 분산과 비슷하게 유지
- 양방향 신호 흐름 설계
- 오토인코더 등에 적합

He (Kaiming)

For ReLU / Leaky ReLU

He 초기화 (2015)

ReLU 활성화 함수에 최적화된 현대 딥러닝의 표준 방법입니다. ReLU가 음수를 0으로 만들어 절반의 뉴런이 비활성화되는 특성을 고려합니다.

공식: $\sqrt{2/n_{in}}$

- 더 큰 초기 가중치로 신호 보존
- 일방통행 신호 흐름 설계
- CNN, ResNet 등 현대 구조에 필수

- ☐ 실전 선택 가이드: CNN + ReLU 조합(가장 흔한 경우)에는 He 초기화를, LSTM/GRU에는 직교 초기화를, Batch Normalization을 사용할 때도 He 초기화가 권장됩니다.

1. **CNN + ReLU** (가장 흔한 케이스)

```
nn.init.kaiming_normal_(layer.weight, mode='fan_in', nonlinearity='relu')
```

2. Linear + Tanh

```
nn.init.xavier_normal_(layer.weight)
```

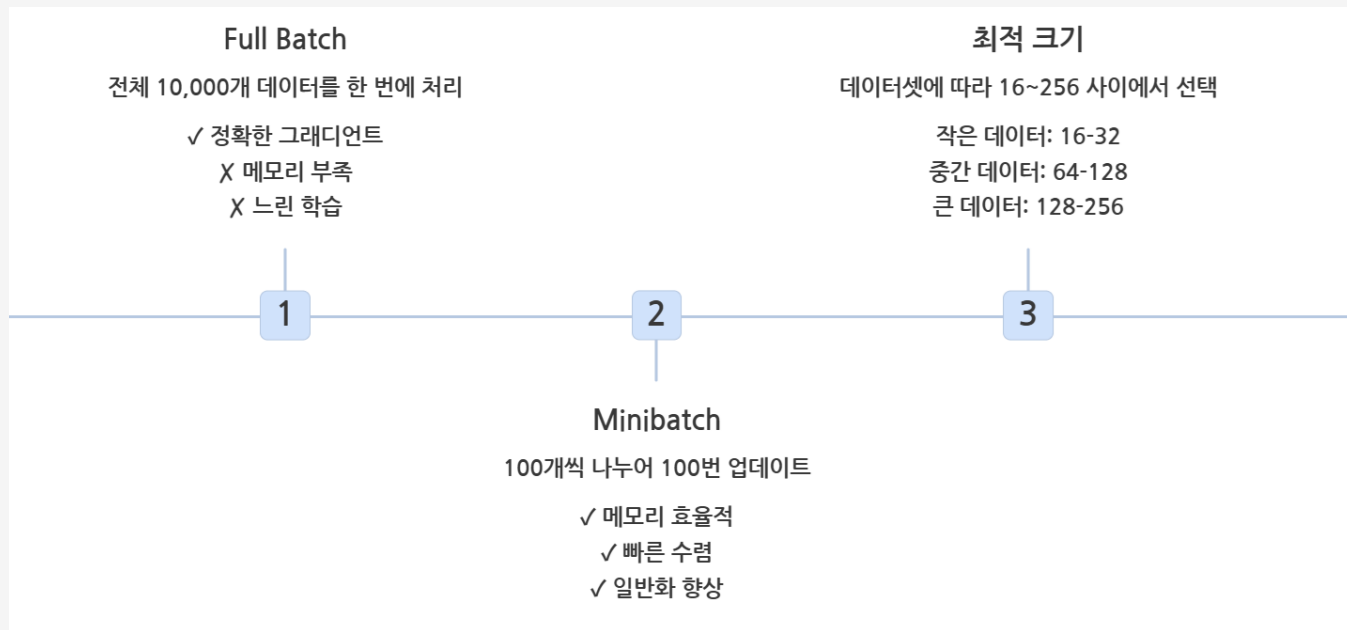
3. LSTM/GRU

```
nn.init.orthogonal_(layer.weight) # 직교 초기화
```

4. **Batch Normalization** 사용 시

```
# 초기화의 중요도가 감소하지만 여전히 He 사용 권장
```


전체 데이터를 한 번에 학습하는 대신 작은 묶음(minibatch)으로 나누어 학습하는 것이 현대 딥러닝의 핵심 전략입니다. 이는 메모리 효율성과 학습 속도를 동시에 개선합니다.



Full Batch (전체 데이터를 한 번에 처리)**장점**

- 정확한 그래디언트 계산
- 부드러운 수렴

단점

- 메모리 부족 (큰 데이터셋 불가능)
- 느린 학습 (1 epoch에 1번만 업데이트)
- 지역 최소값에 갇힐 위험

Mini Batch(데이터를 작은 묶음으로)

예시: 10,000개 데이터를 batch size 100으로

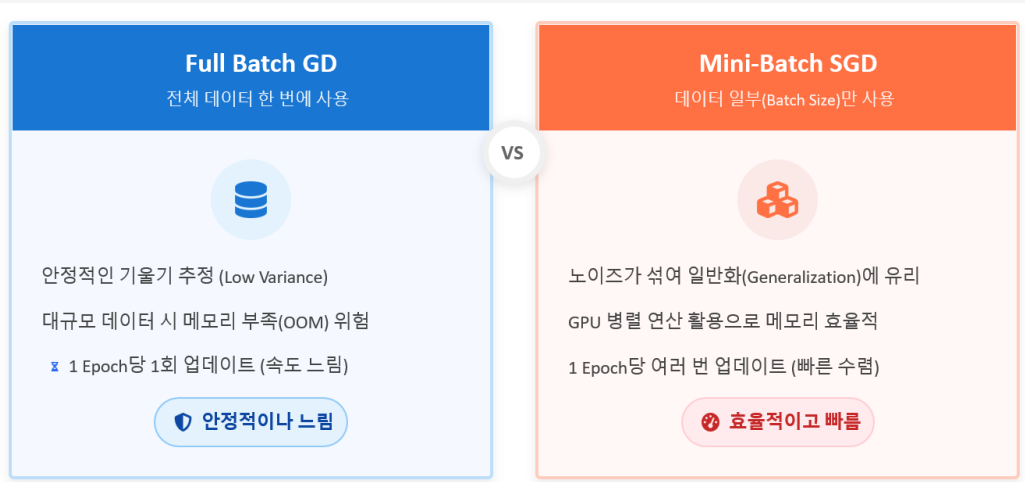
Batch 1 (샘플 1-100) → 업데이트 1

Batch 2 (샘플 101-200) → 업데이트 2

...

Batch 100 (샘플 9901-10000) → 업데이트 100

1 epoch = 100번 업데이트



배치 사이즈는 어떻게 선택해야 하나요?

작은 배치 사이즈(16-32)

장점

- 정규화 효과 (노이즈가 많아 일반화 좋음)
- 메모리 절약

단점

- 불안정한 학습 (그래디언트 노이즈)
- 느린 계산 (GPU 활용도 낮음)

큰 배치 사이즈(256-1024)

장점

- 안정적 학습 (정확한 그래디언트)
- 빠른 계산 (GPU 병렬화)

단점

- 일반화 성능 저하
- 메모리 부족 위험

BatchNormalization

Internal Covariate Shift 현상 문제

>> 각 층의 입력 분포가 학습 중 계속 변화하는 현상

작동원리

[Before BN]

Layer 1 출력: 평균=5, 분산=100 (매우 큼)

↓

Layer 2 입력: 분포가 너무 넓음 → 학습 어려움

[After BN]

Layer 1 출력: 평균=5, 분산=100

↓ Batch Normalization 적용

↓ 정규화: $(x - \text{평균}) / \sqrt{\text{분산}}$

↓ 스케일: $\gamma \times \text{정규화값} + \beta$

Layer 2 입력: 평균 ≈ 0 , 분산 ≈ 1 (안정적)

BN 효과

1) Gradinet Flow 개선

[Without BN]

Layer 1 gradient: 0.001 (소실)

Layer 5 gradient: 0.05

Layer 10 gradient: 0.1

[With BN]

Layer 1 gradient: 0.03 (개선)

Layer 5 gradient: 0.04

Layer 10 gradient: 0.05

→ 모든 층이 고르게 학습

2) 학습률 증가 가능

Without BN: 학습률 0.001 (안전하게 작게)

With BN: 학습률 0.01 (10배 증가 가능)

→ 빠른 학습

**내부 공변량 변화 해결**

학습 중 이전 층의 파라미터
변화로 인해
입력 분포가 계속 바뀌는 현상을 방지

**강력한 학습 안정성**

학습 속도(수렴) 가속화,
초기화 값에 대한 민감도 감소,
오버피팅 억제 효과

3) 초기화 민감도 감소

Without BN: 초기화 잘못되면 학습 실패

With BN: 초기화 영향 감소 (분포 정규화)

4) 정규화 효과(drop-out 대체 가능)

각 Minibatch마다 다른 통계 사용 → 노이즈 효과 → 과적합 방지

[실제 사용 가이드]

올바른 순서

x = Linear(x)

x = BatchNorm(x)

x = ReLU(x)

또는

x = Linear(x)

x = ReLU(x)

x = BatchNorm(x) # ReLU 후에도 가능

Batch Normalization은 RNN 모델에는 부적합

- 시퀀스 길이가 다르면 배치 정규화가 어려움

Layer Normalization

- 배치가 아닌 각 샘플의 특징 차원에 대해 정규화
- RNN, Transformer에 적합

